Step 1: User Interface Initialization (HTML & CSS)

1. Load the base HTML structure using base.html.
2. Apply Bootstrap for responsive layout and main.css for custom styling.
3. Hide image preview section, loader animation, and result text initially.
4. Display a file upload button that allows users to select an image of a tomato leaf.

Step 2: Image Upload and Preview (JavaScript – main.js)

1. Wait until the web page is fully loaded (document.ready).
2. When the user selects an image file:
    - Read the image using FileReader.
    - Display the uploaded image in the preview section.
    - Show the "Predict" button.
3. Ensure only valid image formats (.jpg, .jpeg, .png) are accepted.

Step 3: Prediction Request Handling (Frontend)

1. When the user clicks the **Predict** button:
    - Collect the uploaded image using a FormData object.
    - Hide the predict button.
    - Display a loading spinner to indicate processing.
2. Send the image to the Flask backend using an AJAX **POST** request at /predict.

Step 4: Flask Application Setup (Backend – app.py)

1. Initialize the Flask application.
2. Configure TensorFlow GPU memory usage for optimized performance.
3. Load the trained deep learning model (model_inception.h5) at server startup.
4. Define routes:
    - / → Loads the main webpage.
    - /predict → Handles image upload and prediction.

Step 5: Image Reception and Storage (Backend)

1. Receive the uploaded image file from the POST request.
2. Secure the file name using secure_filename.
3. Save the image inside the uploads directory for processing.

Step 6: Image Preprocessing

1. Load the saved image using Keras utilities.
2. Resize the image to **224 × 224 pixels**.
3. Convert the image to a NumPy array.
4. Normalize pixel values by dividing by 255.
5. Expand image dimensions to match model input shape.

Step 7: Disease Prediction Using Deep Learning Model

1. Pass the preprocessed image to the trained Inception-V3 model.
2. Obtain prediction probabilities for all disease classes.
3. Identify the class with the highest probability using argmax.

Step 8: Disease Label Mapping

1. Convert the predicted class index into a human-readable disease name.
2. Map prediction output to diseases such as:
    - Tomato Early Blight
    - Tomato Bacterial Spot
    - Tomato Mosaic Virus
    - Healthy Leaf
3. Generate a meaningful prediction message.

Step 9: Sending Result to Frontend

1. Return the predicted disease result as a text response to the AJAX call.
2. Stop loader animation.
3. Display the disease prediction result on the webpage.

Step 10: Model Training Using Transfer Learning (Inception-V3)

1. Load the pre-trained Inception-V3 model with ImageNet weights.
2. Freeze all base layers to retain learned features.
3. Add custom layers:
    - Flatten layer
    - Fully connected Dense layer with Softmax activation
4. Train the model on tomato leaf disease dataset.
5. Save the trained model as model_inception.h5 for deployment.

Step 11: System Completion

1. The system successfully predicts tomato leaf diseases from uploaded images.
2. The result is displayed in real time through a web-based interface.
3. The application integrates deep learning, Flask backend, and frontend technologies.

End of Algorithm